Convolutional Neural Networks

For Image Classification

Muhammad Anthar Jailani Putra (11220940000029)
Reyhan Maulana A (11220940000048)

A TIMELINE ALWAYS WORKS WELL

Load Dataset

Dataset CIFAR 10

Costum CNN & ResNet18

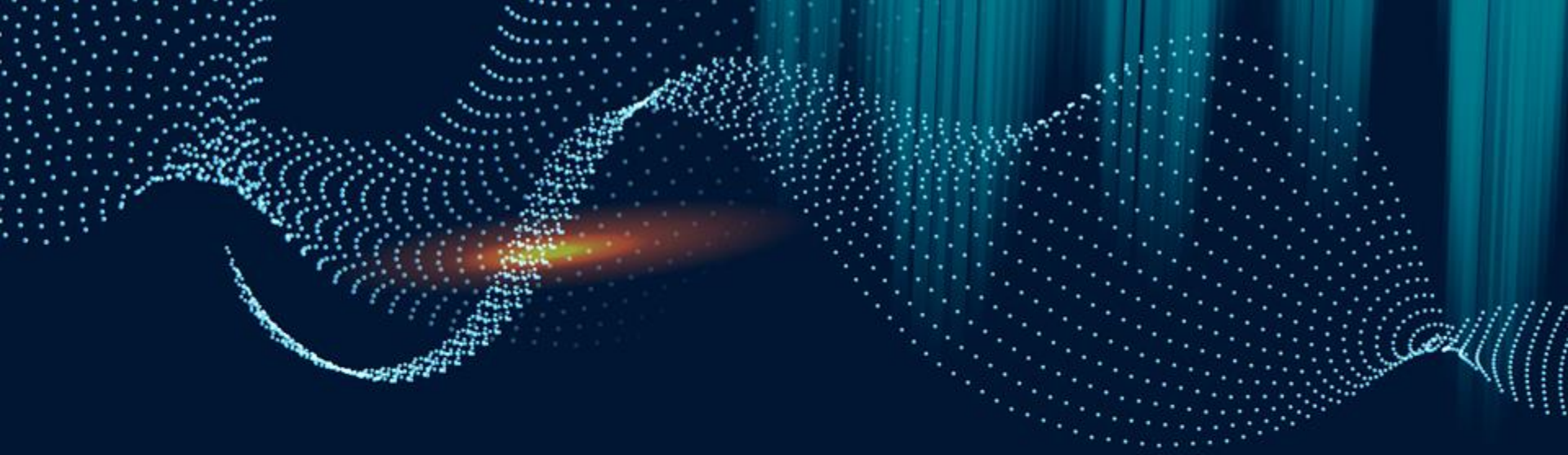
Model

Training

Hyperparameter Tuning

Evaluation Metrics & Feature Maps

Evaluation



01 Load Dataset

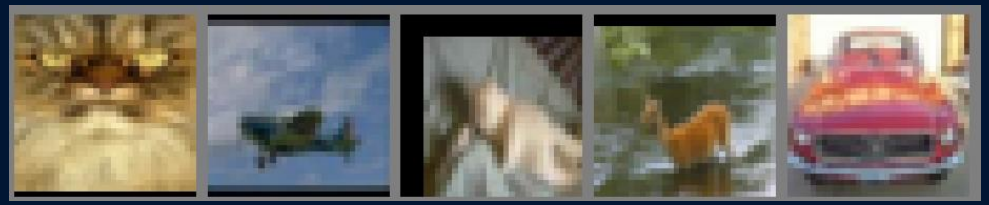
CIFAR 10

CIFAR-10 adalah dataset gambar berwarna berukuran 32x32 piksel yang terdiri dari 60.000 gambar dalam 10 kelas berbeda.

class : airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck.

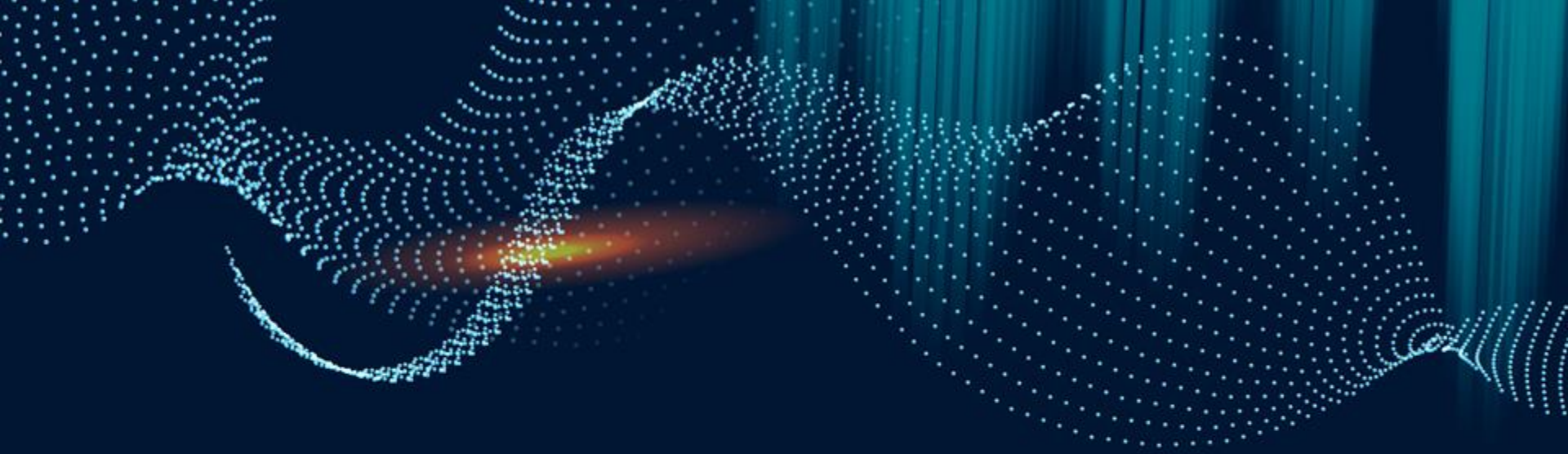
IMAGE PREPROCESSING

- Random Horizontal Flip
- Random Crop
- Transform to tensor
- Normalize ((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))



SPLITING DATASET

- Trainset : 45.000
- Validationset : 5.000
- Testset : 10.000



02 Model

ResNet 18

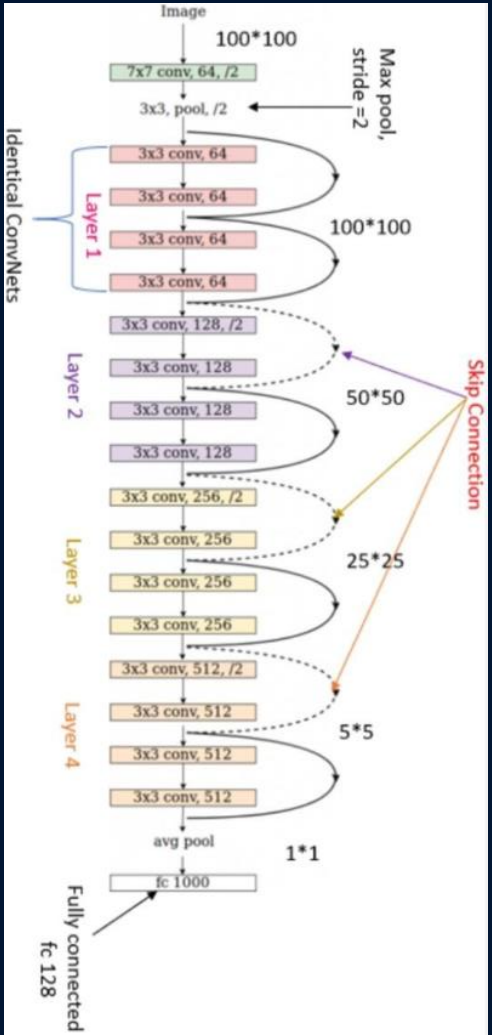
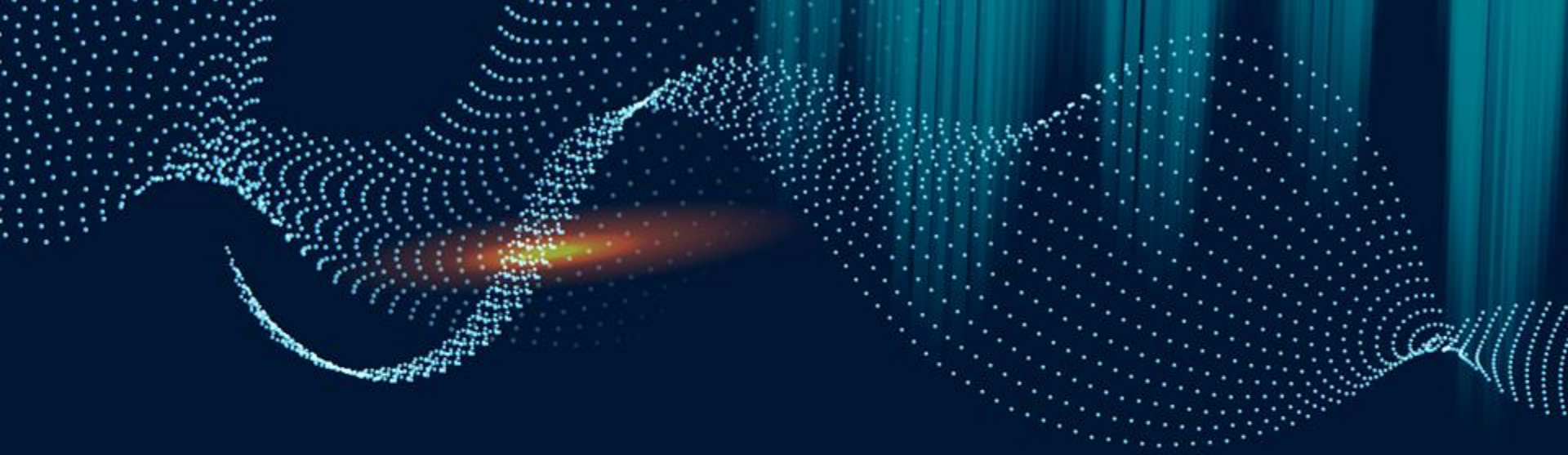


Image
100*100
7x7 conv, 64, /2
3x3, pool, /2
Max pool, stride =2
Identical ConvNets
Layer 1
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
3x3 conv, 64
100*100
3x3 conv, 128, /2
Layer 2
3x3 conv, 128
3x3 conv, 128
3x3 conv, 128
50*50
Skip Connection
3x3 conv, 256, /2
Layer 3
3x3 conv, 256
3x3 conv, 256
3x3 conv, 256
25*25
3x3 conv, 512, /2
Layer 4
3x3 conv, 512
3x3 conv, 512
3x3 conv, 512
5*5
avg pool
1*1
fc 1000
Fully connected
fc 128

Costum CNN

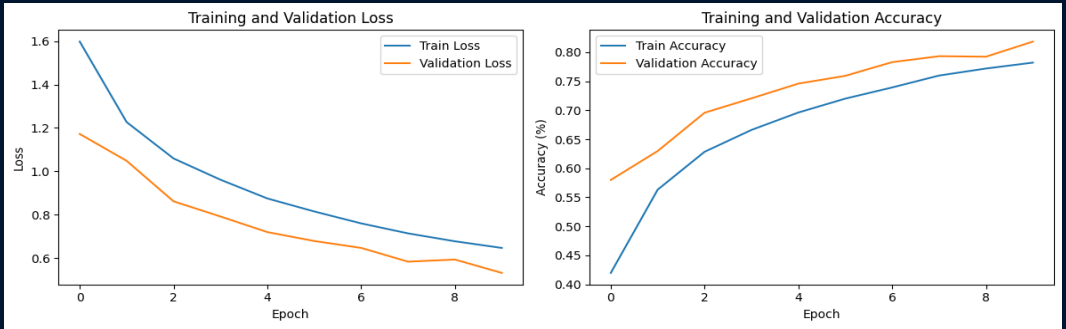
```python
# Define the enhanced CNN architecture
class CustomCNN(nn.Module):
    def __init__(self):
        super(CustomCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)  # Increased filters to 64
        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)  # Increased to 128
        self.conv3 = nn.Conv2d(128, 256, 3, padding=1)  # Increased to 256
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(256 * 4 * 4, 1024)  # Adjusted for new filter size
        self.fc2 = nn.Linear(1024, 10)
        self.dropout = nn.Dropout(0.4)  # Reduced dropout to 0.4
        self.batchnorm1 = nn.BatchNorm2d(64)
        self.batchnorm2 = nn.BatchNorm2d(128)
        self.batchnorm3 = nn.BatchNorm2d(256)

    def forward(self, x):
        x = self.pool(F.relu(self.batchnorm1(self.conv1(x))))
        x = self.pool(F.relu(self.batchnorm2(self.conv2(x))))
        x = self.pool(F.relu(self.batchnorm3(self.conv3(x))))
        x = x.view(-1, 256 * 4 * 4)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x
```



03 Training

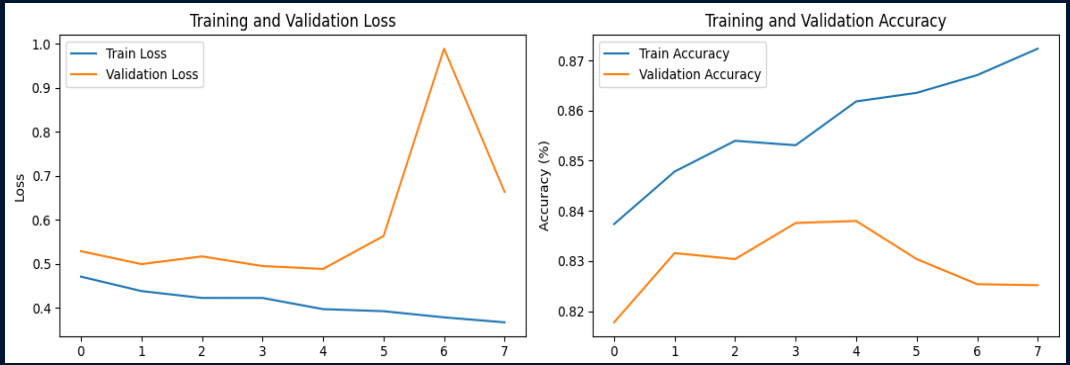
Hyperparameter Tuning

Costum CNN

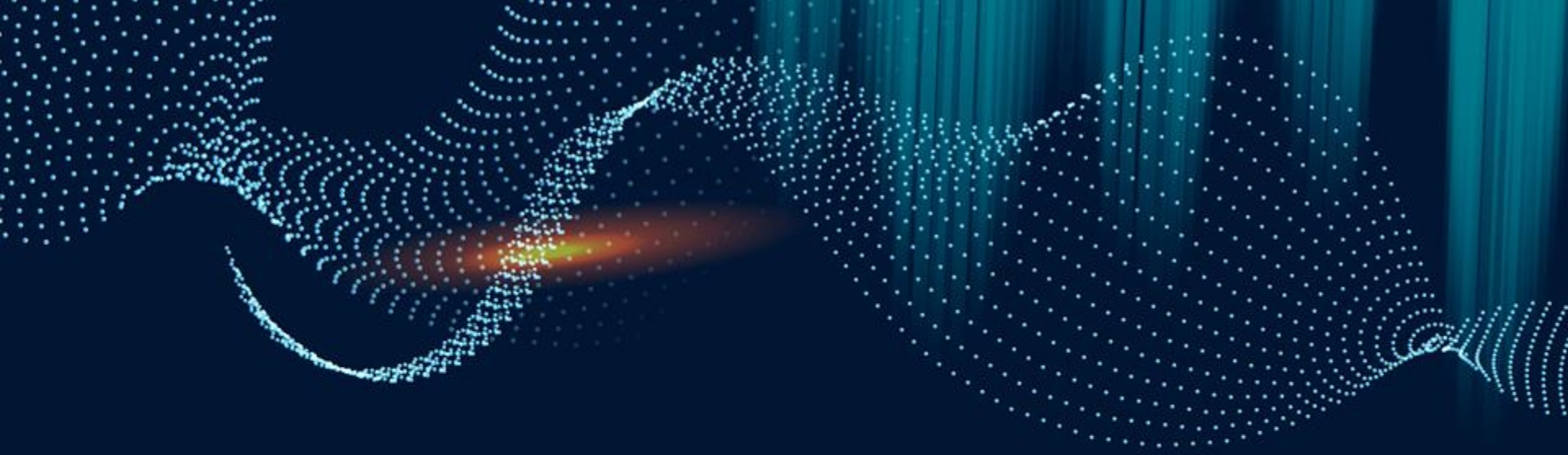



```
🔍 Tuning: lr=0.001, batch_size=32
Epoch 1/10 | Train Loss: 1.5975, Train Acc: 41.98% | Val Loss: 1.1724, Val Acc: 58.00%
Epoch 2/10 | Train Loss: 1.2270, Train Acc: 56.33% | Val Loss: 1.0488, Val Acc: 62.98%
Epoch 3/10 | Train Loss: 1.0597, Train Acc: 62.85% | Val Loss: 0.8624, Val Acc: 69.58%
Epoch 4/10 | Train Loss: 0.9619, Train Acc: 66.62% | Val Loss: 0.7926, Val Acc: 72.06%
Epoch 5/10 | Train Loss: 0.8756, Train Acc: 69.62% | Val Loss: 0.7208, Val Acc: 74.60%
Epoch 6/10 | Train Loss: 0.8158, Train Acc: 72.01% | Val Loss: 0.6797, Val Acc: 75.94%
Epoch 7/10 | Train Loss: 0.7609, Train Acc: 73.94% | Val Loss: 0.6480, Val Acc: 78.30%
Epoch 8/10 | Train Loss: 0.7149, Train Acc: 75.99% | Val Loss: 0.5848, Val Acc: 79.32%
Epoch 9/10 | Train Loss: 0.6785, Train Acc: 77.20% | Val Loss: 0.5944, Val Acc: 79.24%
Epoch 10/10 | Train Loss: 0.6479, Train Acc: 78.22% | Val Loss: 0.5330, Val Acc: 81.84%
```

Resnet18




```
🔍 Tuning: lr=0.003, batch_size=64
Epoch 1/10 | Train Loss: 0.4708, Train Acc: 83.74% | Val Loss: 0.5287, Val Acc: 81.78%
Epoch 2/10 | Train Loss: 0.4379, Train Acc: 84.78% | Val Loss: 0.4992, Val Acc: 83.16%
Epoch 3/10 | Train Loss: 0.4223, Train Acc: 85.40% | Val Loss: 0.5167, Val Acc: 83.04%
Epoch 4/10 | Train Loss: 0.4223, Train Acc: 85.31% | Val Loss: 0.4948, Val Acc: 83.76%
Epoch 5/10 | Train Loss: 0.3969, Train Acc: 86.18% | Val Loss: 0.4882, Val Acc: 83.80%
Epoch 6/10 | Train Loss: 0.3922, Train Acc: 86.35% | Val Loss: 0.5631, Val Acc: 83.04%
Epoch 7/10 | Train Loss: 0.3782, Train Acc: 86.71% | Val Loss: 0.9890, Val Acc: 82.54%
Epoch 8/10 | Train Loss: 0.3669, Train Acc: 87.23% | Val Loss: 0.6633, Val Acc: 82.52%
Early stopping at epoch 8
```

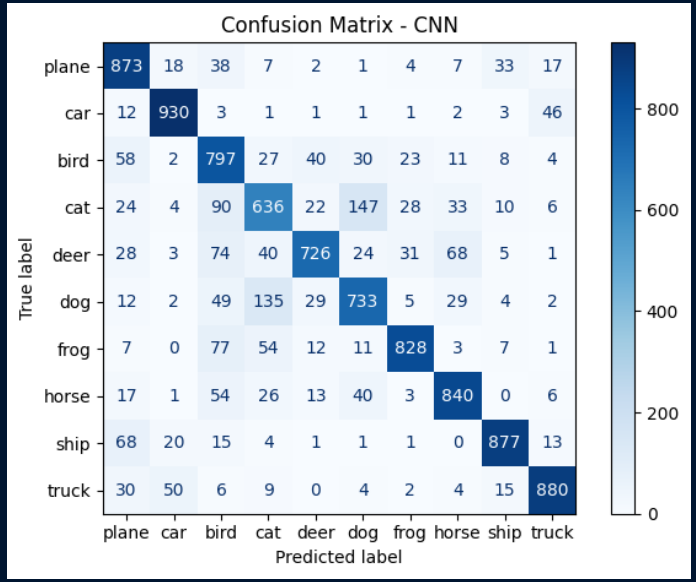


04 Evaluasi

Confusion Matriks

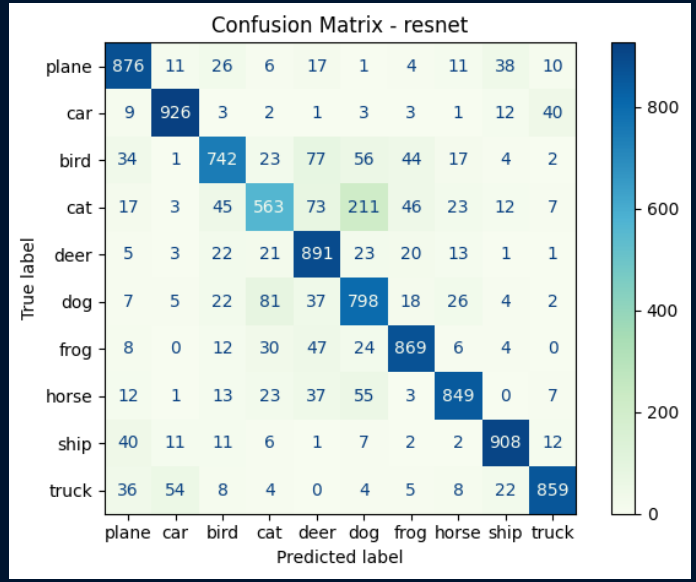
Costum CNN

	Accuracy	Precision (macro)	Recall (macro)	F1 Score (macro)
CNN	0.812	0.816304	0.812	0.812418



Resnet18

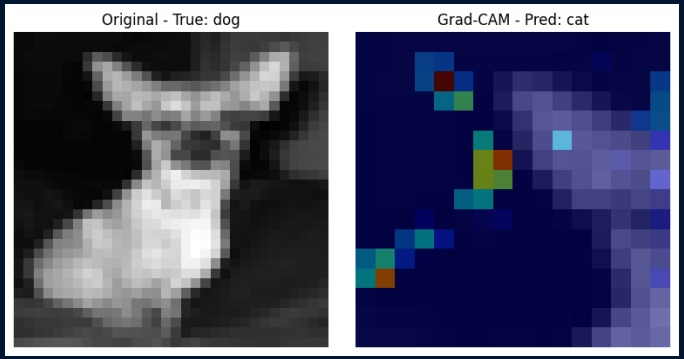
	Accuracy	Precision (macro)	Recall (macro)	F1 Score (macro)
resnet	0.8281	0.830592	0.8281	0.826665

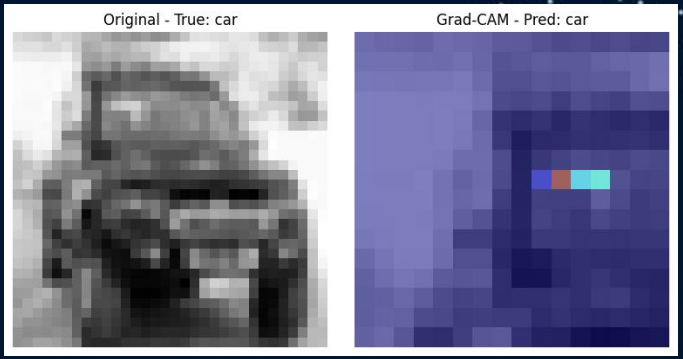


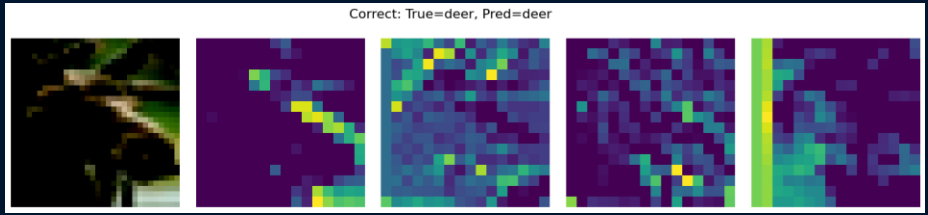
cat : -73
deer : +165
dog : +65
ship : +31

Feature Maps

Costum CNN

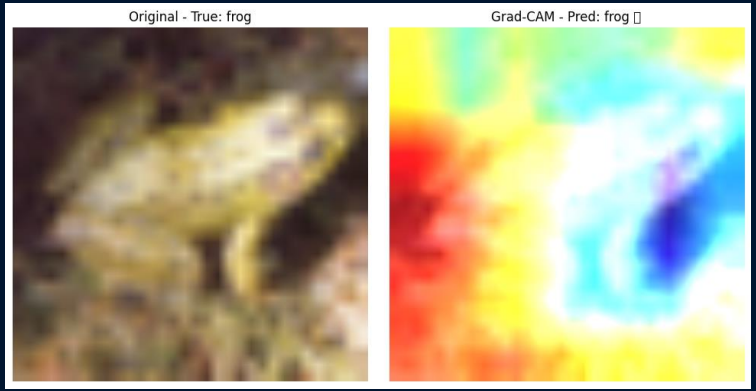


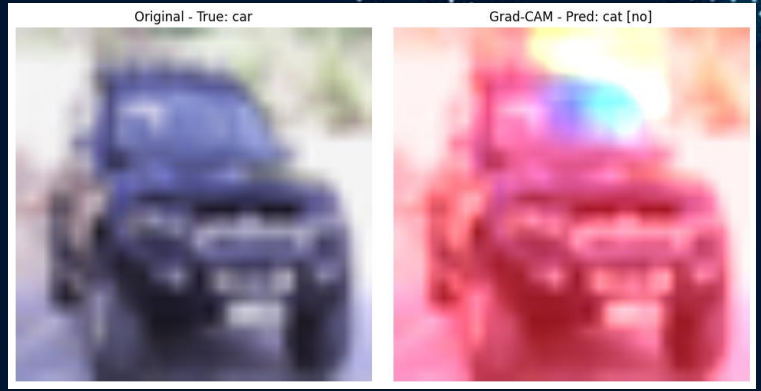


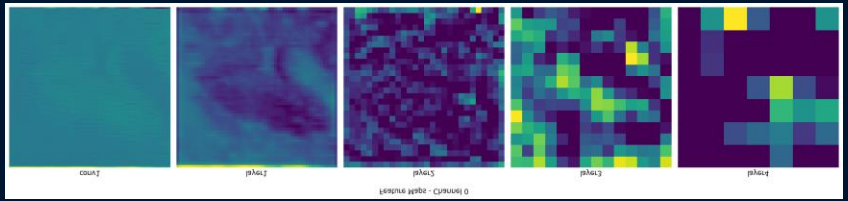


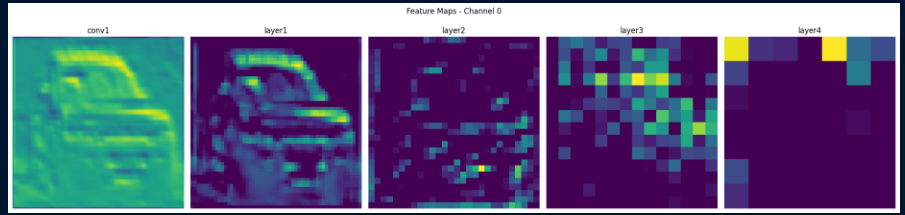
Feature Maps

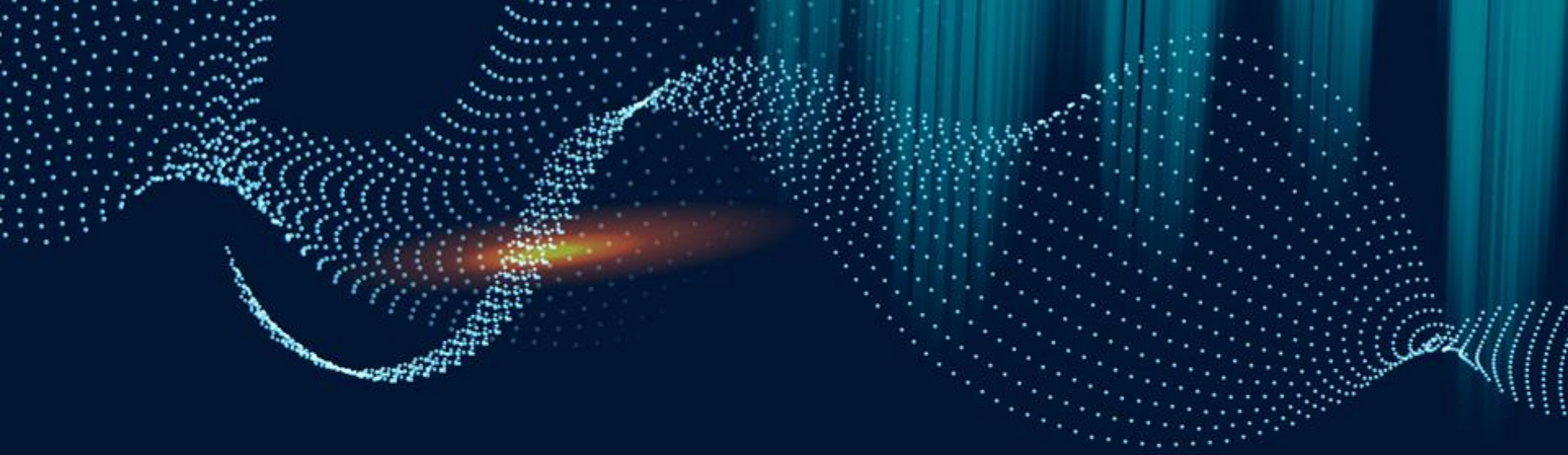
Resnet18











THANK YOU